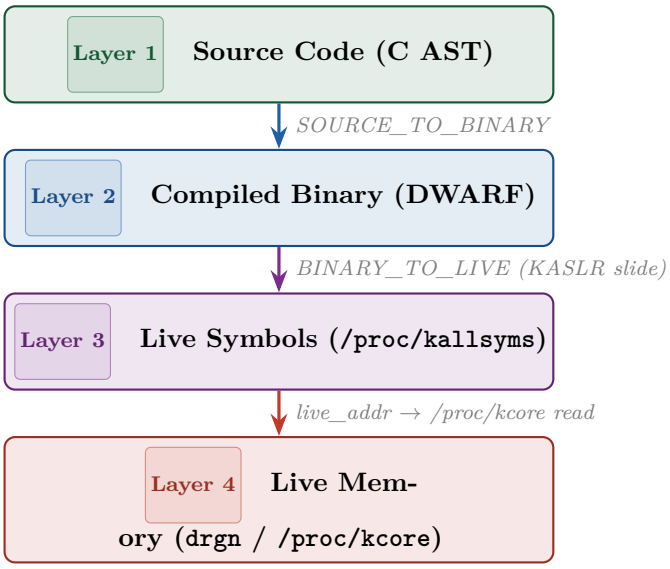
Kernel-Talk

A Digital Twin for the Linux Kernel

Architecture & System Design Reference



> *"Vector search tells you what code looks like what you asked about.*
> *Graph traversal tells you what it depends on.*
> *drgn tells you what is actually happening right now."*

Lee Ostadi

Graduate Researcher — Machine Learning & Neural Networks

April 18, 2026

Contents

1 System Overview

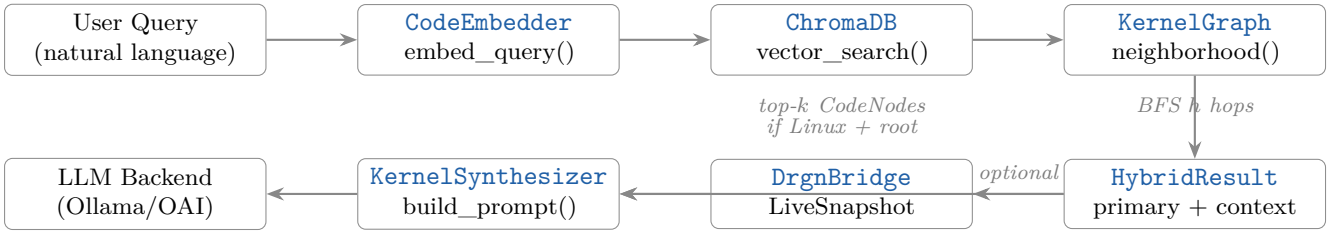
Kernel-Talk is a **Digital Twin** for the Linux kernel: a system that simultaneously holds the static source structure of the kernel (its theory) and its live runtime state (its reality), fusing them through language models to answer questions that neither source code search nor runtime debugging can answer alone.

The central claim is that understanding why a kernel behaviour occurs requires traversing *four distinct representations* of the same artefact:

1. **C source code** — the human-readable expression of intent, parsed into a semantic graph by tree-sitter.
2. **Compiled binary** (vmlinux DWARF) — the compiled truth: function address ranges, struct field byte offsets, inlining decisions.
3. **Live symbol table** (`/proc/kallsyms`) — the KASLR-adjusted addresses of every exported kernel symbol *in the running kernel*.
4. **Live memory** (drgn / `/proc/kcore`) — actual struct field values at runtime, without stopping execution.

These are not independent datastores. They are connected by precise mathematical relationships (KASLR slide, DWARF offset tables) encoded as typed edges in a unified **KernelGraph**. The architecture is designed so that any question can be traced from a user's natural-language query through all four layers to a grounded, citable answer.

1.1 End-to-End Query Path



The query pipeline is a two-stage retrieval followed by generative synthesis. Stage 1 (*retrieve*) is entirely offline-capable—it uses pre-built vector and graph indices. Stage 2 (*synthesize*) calls an LLM with a structured prompt containing the retrieved code context and optional live state. The LLM backend is pluggable: Ollama (default, offline), OpenAI-compatible, or Anthropic.

2 Repository Structure

kernel-talk/ -- top-level layout

```
kernel-talk/
|-- cli/
|   `-- ktalk.py              # CLI entry point (argparse commands)
|-- core/
|   |-- mirror/               # The Mirror: static code twin
|   |   |-- parser.py         # KernelParser, CodeNode, tree-sitter
    queries
|   |   |-- graph.py          # KernelGraph, EdgeType, GraphContext
|   |   |-- embedder.py       # CodeEmbedder (CodeBERT), chunk_text
|   |   `-- store.py          # KernelStore: unified retrieval
   interface
|   |-- dwarf/
|   |   `-- bridge.py         # DwarfBridge, BinarySymbol,
   StructLayout
|   |-- probe/
|   |   |-- kallsyms.py       # KallsymsBridge, KallsymsEntry,
   KASLR slide
|   |   `-- drgn_bridge.py    # DrgnBridge, LiveSnapshot,
   ProcessSnapshot
|   `-- synthesis/
|       `-- synthesizer.py    # KernelSynthesizer, build_prompt
|-- training/
|   |-- mine.py               # TripletMiner: git history -> JSONL
|   |-- bm25.py               # BM25Index: hard negative enrichment
|   |-- dataset.py            # TripletDataset: curriculum-aware
   DataLoader
|   `-- train_biencoder.py    # BiEncoder fine-tuning with InfoNCE
|-- tools/
|   `-- xray.py               # Filesystem X-Ray (/sys, /proc
   resolution)
`-- tests/
    |-- test_graph.py         # KernelGraph unit tests
    |-- test_persistence.py   # Save/load roundtrip tests
    |-- test_store.py         # KernelStore integration tests
    `-- test_training.py      # 50 tests: BM25, mining, curriculum
```

3 The Mirror — Static Code Twin

The Mirror is Layer 1 of the Digital Twin. It ingests the entire Linux kernel C source tree and produces two complementary indices: a **vector index** (ChromaDB) for semantic similarity search, and a **knowledge graph** (NetworkX MultiDiGraph) for structural context expansion. Both are built from the same atomic unit: the `CodeNode`.

3.1 The `CodeNode` — Atomic Unit

`core/mirror/parser.py -- CodeNode`

```python
@dataclass
class CodeNode:
    id: str             # "kernel/sched/fair.c::tg_throttle_up"
    node_type: str      # function | struct | union | enum | macro | file
    symbol_name: str    # bare C identifier
    file_path: str      # relative to kernel root
    line_start: int     # 1-indexed
    line_end: int
    code: str           # raw C source text
    docstring: str      # preceding /** ... */ block comment
    calls: list[str]    # callee names (-> CALLS graph edges)
    uses_structs: list[str]  # struct/union refs (-> USES_STRUCT edges)
    includes: list[str]      # #include paths (file nodes only)
```

A `CodeNode` is simultaneously:

- A **graph node** — its `id` is the vertex key in `KernelGraph`.
- A **vector document** — its `embedding_text()` is stored in ChromaDB.
- A **training unit** — it is the target of BM25 and bi-encoder retrieval.

The `embedding_text()` method concatenates `docstring + "// type: name in path" + code`. The docstring prepend is critical: kernel developers write highly information-dense block comments that capture *intent*, while the code body captures *mechanism*. Fusing both gives the CodeBERT embedding far richer semantic signal than code alone.

3.2 `KernelParser` — AST Extraction

`KernelParser` (`core/mirror/parser.py`) walks the kernel source tree and extracts `CodeNode` objects via tree-sitter C queries. It extracts six node types, each with a dedicated S-expression query:

Node type	tree-sitter tag	Graph edges populated	Notes
`function`	`function_definition`	`CALLS, USES_STRUCT, DEFINED_IN`	Primary retrieval target
`struct`	`struct_specifier`	`DEFINED_IN`	Layout decoded by DWARF
`union`	`union_specifier`	`DEFINED_IN`	Anonymous unions flattened (F
`enum`	`enum_specifier`	`DEFINED_IN`	Constants
`macro`	`preproc_def`	`DEFINED_IN`	Non-trivial values only
`file`	(synthetic)	`INCLUDES`	Header dependency graph

Key design decisions:

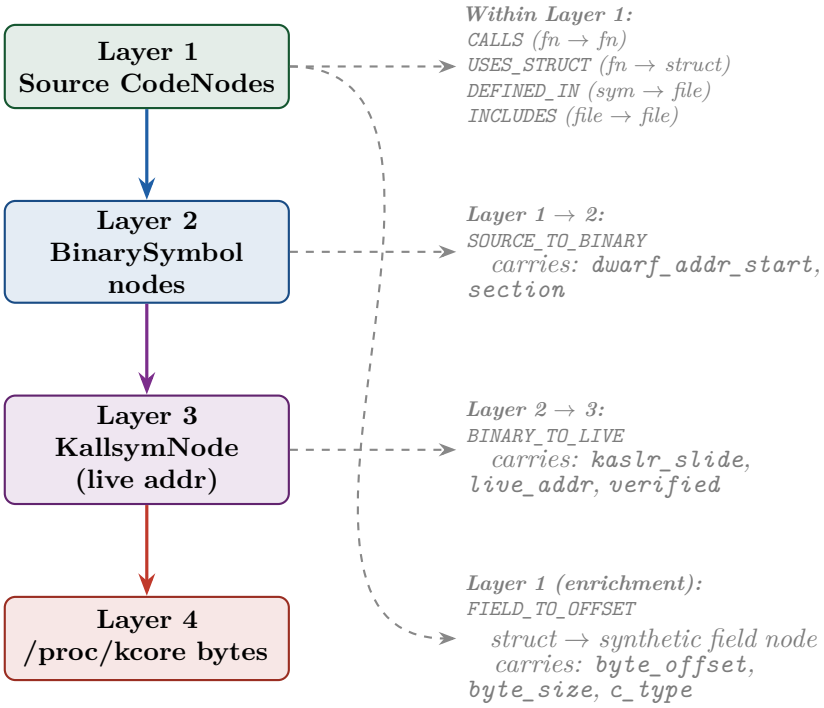
- **CORE_STRUCTS** is a `frozenset` (immutable, thread-safe). Every parsed struct/union name is registered into this set dynamically to enable `USES_STRUCT` edge detection in later files.
- **SKIP_PATTERNS** filters generated files (`.mod.c`, `.tmp.`), test fixtures, and tools — noise that degrades embedding quality.

- **Size cap:** files exceeding 2 MB are skipped. These are typically generated tables (ID tables, firmware blobs) not useful for retrieval.

3.3 `KernelGraph` — Structural Knowledge Graph

`KernelGraph` (`core/mirror/graph.py`) is a `nx.MultiDiGraph` — directed, with multiple edge types allowed between the same pair of nodes. It serves as the structural backbone of the entire Digital Twin.

3.3.1 Edge Types and Layer Crossings



3.3.2 Index Structures for O(1) Edge Resolution

Naive edge resolution ($O(N^2)$ for INCLUDES matching) was replaced by three internal indices built lazily during `add_node()`:

`_symbol_index dict[name, list[node_id]]` — enables $O(1)$ lookup of all definitions of a symbol name. Required for CALLS and USES_STRUCT edges (a name may have multiple static definitions in different compilation units).

`_file_index dict[path, node_id]` — direct map from file path to file-node ID, for DEFINED_IN edges.

`_includes_suffix_index dict[suffix, list[node_id]]` — Linux `#include` paths are relative (e.g. `"linux/sched.h"`). We index every trailing suffix of each file path so that resolution of any relative include is $O(1)$ (F-5 fix).

3.3.3 Serialisation Contract (F-6, F-13)

`save()` serialises *only* Layer 1 (CodeNode) nodes to GraphML. Layer 2 (BinarySymbol) and Layer 3 (live-address dict) nodes are omitted because they are recomputable via `ktalk twin` and their types (large integers, nested dicts) are incompatible with GraphML's string-typed attribute model. A `schema_version` integer is stamped on the graph-level metadata so `load()` can detect stale saves.

3.4 `CodeEmbedder` — Dense Retrieval Embeddings

`CodeEmbedder` (`core/mirror/embedder.py`) wraps `microsoft/codebert-base` (125M parameters, 768-dim output) from HuggingFace via `sentence-transformers`.

Aspect	Design	Rationale
Model choice	`codebert-base`	Pre-trained on docstring↔code pairs
Context limit	512 tokens / ≈1800 chars	BERT architectural constraint
Long-form handling	Overlapping chunks + mean-pool	Preserves function signatures
Device auto-select	CUDA > MPS (Apple) > CPU	Maximises available hardware
Output normalisation	L2 unit sphere	Cosine sim ≡ dot product
Query prefix	`"query: {text}"`	Consistent with E5 convention

Long-function chunking: code exceeding the 512-token limit is split on line boundaries into overlapping chunks (450 tokens, 50-token overlap), embedded independently, then mean-pooled and renormalised. This ensures a 500-line kernel function receives a faithful embedding rather than a silently truncated one.

3.5 `KernelStore` — Unified Retrieval Interface

`KernelStore` (`core/mirror/store.py`) is the single public access point for all Mirror data. It owns both a ChromaDB persistent client (vector index) and a `KernelGraph` (structural index), and exposes them through a unified `hybrid_search()` interface.

`KernelStore.hybrid_search() -- dual-retrieval pipeline`

```
def hybrid_search(query, top_k=8, hops=2, ...) -> HybridResult:
    # Step 1: embed query + cosine search in ChromaDB
    vector_results = self.vector_search(query, top_k=top_k)

    # Step 2: BFS expand from vector seeds in KernelGraph
    seed_ids = [r.node.id for r in vector_results]
    graph_ctx = self.graph.neighborhood(seed_ids, hops=hops)

    # Step 3: resolve neighbor CodeNodes, dedup against primary
    context_nodes = [self.graph.get_node(nid) for nid in
                     graph_ctx.neighbor_ids if nid not in
                        primary_ids]

    return HybridResult(primary=vector_results, context=
        context_nodes,
                        graph_ctx=graph_ctx)
```

The `HybridResult` carries:

- **primary** — top-k vector hits, ranked by cosine similarity.
- **context** — graph-expanded neighbours (callers, callees, used structs), structurally related but not necessarily high-similarity.
- **graph_ctx** — the raw induced subgraph for visualisation/inspection.

`all_nodes()` merges primary and context deduplicated; the output is rendered to a structured text block for the LLM via `to_context_text()`.

4 Layer 2 — The DWARF Bridge

`DwarfBridge` (`core/dwarf/bridge.py`) elevates Kernel-Talk from a code search engine to a true Digital Twin by parsing the compiled vmlinux ELF with DWARF debug information (via `pyelftools`).

4.1 Data Model

`BinarySymbol` A compiled function or variable with fields: `addr_start`, `addr_end` (virtual address range in `.text`), `source_file`, `source_line`, `symbol_type` (`function|inline`).

`StructLayout` The decoded memory layout of a C struct: `total_size` (from `DW_AT_byte_size`) and a dict of `FieldInfo` entries giving each field's byte offset, size, and C type. Used to decode raw `/proc/kcore` bytes without `drgn`'s type system.

`LineEntry` A single DWARF line number program entry mapping a virtual address to a source file, line, and column. Powers `addr_to_line()` (the `addr2line` functionality).

4.2 Key Algorithms

Address lookup (O(log N)) All parsed functions are stored sorted by `addr_start`. `addr_to_symbol(addr)` uses `bisect.bisect_right` to find the rightmost function whose start $\leq$ addr, then verifies containment in $[lo\_pc, hi\_pc)$. Same approach for line table lookup.

Anonymous union flattening (F-16) Kernel structs like `task_struct` contain anonymous `union`/`struct` members with no DWARF name. `_flatten_anon_members()` recursively walks such DIEs and merges their fields into the parent `StructLayout` with corrected absolute byte offsets. This ensures every field (including those nested inside anonymous unions) is directly accessible via `struct_field_offset()`.

Cache key (F-14) The pickle cache key is `mtime + file_size + CRC32(first 64KB)`. `mtime` alone is fragile (filesystem truncates to 1-second precision; copies preserve mtime). Adding size catches same-mtime / different-content collisions; CRC32 of the ELF header catches same-mtime + same-size collisions essentially perfectly at < 1 ms cost.

DW_AT_high_pc duality DWARF 4+ allows `DW_AT_high_pc` to be either an absolute address (form class `address`) or a byte offset from `DW_AT_low_pc` (form class `constant`). The parser detects the form class via `describe_form_class()` and handles both.

5 Layer 3 — Live Symbol Table

`KallsymsBridge` (`core/probe/kallsyms.py`) reads and indexes `/proc/kallsyms` — the running kernel's exported symbol table with KASLR-adjusted addresses.

5.1 KASLR Slide Computation

The critical linking operation between Layer 2 (DWARF, pre-KASLR) and Layer 3 (live, post-KASLR) is:

$$\text{live\_address} = \text{dwarf\_address} + \underbrace{(\text{kallsyms\_addr}(\text{anchor}) - \text{dwarf\_addr}(\text{anchor}))}_{\text{KASLR slide}} \quad (1)$$

Multiple anchor symbols are used (scheduler entry points, `_text`, `startup_64`, both pre/post kernel-5.7 variants) and the *median* slide is taken for robustness against per-symbol outliers.

Anchors absent in a given kernel version are silently skipped (F-17: `do_fork` removed in v5.7; F-18: `sys_read` renamed to `__x64_sys_read` on x86-64).

5.2 Data Model and Indices

```
@dataclass
class KallsymsEntry:
    address: int     # KASLR-adjusted virtual address
    sym_type: str    # T/t (text), D/d (data), B/b (bss), A (absolute)
        ...
    name: str
    module: str      # non-empty if from a loaded kernel module
```

Three indices are maintained: `_by_name` (dict[name, list[entry]]), `_by_addr` (dict[addr, entry]), and `_sorted_addrs` (sorted list for binary-search nearest-symbol lookup — the `addr2sym` direction).

6 Layer 4 — Live Memory Probe

`DrgnBridge` (`core/probe/drgn_bridge.py`) is the Theory/Reality interface. It uses `drgn` (Meta's programmable kernel debugger) to read live kernel memory via `/proc/kcore` without stopping execution.

Safety & Access Requirements

- Linux only (reads `/proc/kcore`)
- Root or `CAP_SYS_PTRACE`
- Kernel compiled with `CONFIG_PROC_KCORE=y` (most distros)
- **Read-only**: drgn cannot write kernel memory — safe for production

6.1 High-Level Probes

`list_processes(max_tasks)` Walks the kernel's `task_struct.tasks` linked list anchored at `init_task` to enumerate all live processes. Returns `ProcessSnapshot` objects with pid, ppid, comm, state, cpu, and memory/file descriptor pointers. Handles the kernel-5.14 `task.__state` rename (F-20): tries `__state` first, falls back to `state`.

`read_struct(struct_name, address, fields)` Reads a specific kernel struct instance at a hex address via `prog.object()`. Returns a `LiveSnapshot` with each field's value, C type, and byte offset. Any field raising `AttributeError` or `ObjectAbsentError` is silently skipped (forward-compatible across kernel versions).

`read_runqueue(cpu)` Reads `struct rq` for a specific CPU via the `drgn.helpers.linux.cpu_rq` helper. Captures `nr_running`, `nr_switches`, `clock`, and the currently running task's comm/pid. Has separate ImportError guards for the `drgn` module itself and the `cpu_rq` helper (F-22), providing actionable error messages if the drgn version is too old.

6.2 Graceful Degradation

On non-Linux systems (macOS during development), all probe methods return `None` or synthetic mock data rather than crashing. This allows the full static retrieval pipeline to function without a Linux host, and makes unit testing possible without root access.

7 Synthesis — LLM Integration

`KernelSynthesizer` (`core/synthesis/synthesizer.py`) takes a `HybridResult` (static code context) and an optional list of `LiveSnapshot` objects (runtime state), constructs a structured prompt, and calls a configurable LLM backend.

7.1 Prompt Architecture

The prompt is divided into three blocks with a token-budget-aware allocation (F-24):

[SYSTEM] Kernel expert role. Cite file:function. Bridge theory and reality.

[THEORY] Up to 75% of variable token budget. CodeNode blocks with file, lines, docstring, code.

[REALITY] Up to 25% of variable budget. LiveSnapshot field values from drgn.

[QUESTION + INSTRUCTIONS] User query. Explain causality, cite sources, flag theory/reality mismatches.

Token budget is enforced dynamically: fixed overhead (system + query + instructions) is estimated first; the remainder is split 75/25 between code and live-state blocks. If even the first code node exceeds budget, `max_nodes` is halved iteratively.

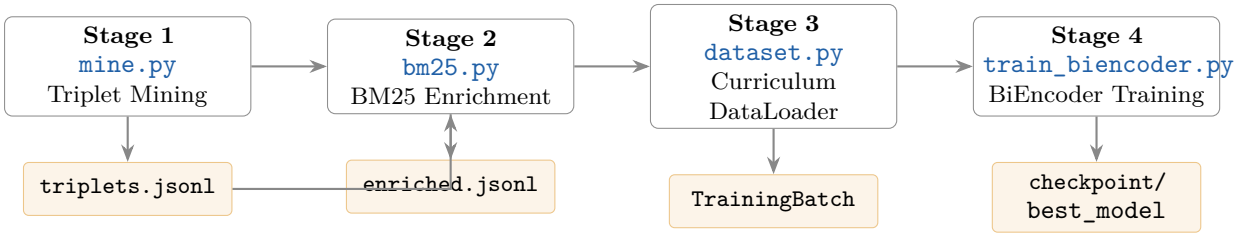
7.2 LLM Backends

Provider string	Backend	Auth	Default model
`ollama:...`	Local Ollama + HTTP fallback	None	`deepseek-coder:6.7b`
`openai:...`	OpenAI / vLLM / LM Studio	`OPENAI_API_KEY`	`gpt-4o`
`anthropic:...`	Anthropic Claude	`ANTHROPIC_API_KEY`	`claude-sonnet-4-6`

Both blocking (`synthesize()`) and streaming (`synthesize_stream()`) modes are supported. Ollama streaming uses raw HTTP line-delimited JSON to avoid requiring the `ollama` Python package.

8 Training Pipeline

The training pipeline converts the Linux kernel git history into a fine-tuned bi-encoder that outperforms the pre-trained CodeBERT baseline on kernel-specific retrieval tasks. It is structured as four sequential, restartable stages:



8.1 Stage 1: Triplet Mining (`mine.py`)

`TripletMiner` exploits a fundamental property of the Linux kernel development process: every commit is a free supervision signal of the form (commit message, changed functions, unchanged functions).

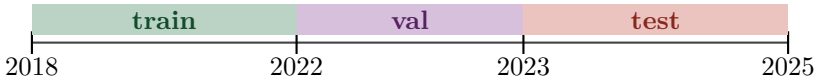
8.1.1 Mining Algorithm

For each commit since a configurable date:

1. **iter_commits()** — stream git log with `-name-only -diff-filter=M` to get modified C/H files per commit.
2. **get_diff_hunks()** — parse unified diff hunk headers (`@@ -old +new_start,count @@`) to extract the first line of each changed region in the new file.
3. **get_function_ranges()** — run `git show SHA:file` to get the file at that commit's parent, then use tree-sitter to extract `(name, start_line, end_line)` for every function. Falls back to a brace-counting regex if tree-sitter is unavailable.
4. **functions_at_lines()** — map hunk start lines to function names by range containment.
5. **is_useful_commit()** — filter merges, reverts, header-only changes, too-short subjects, empty positive sets.

8.1.2 Temporal Split (Critical Correctness)

Triplets are split by *commit date*, never randomly:



Random splits allow data leakage: test commits from the same author / subsystem as training commits are trivially solved by memorisation rather than generalisation. Temporal splits enforce the harder but correct question: *does the model generalise to future kernel changes?*

8.2 Stage 2: BM25 Hard Negative Enrichment (`bm25.py`)

`BM25Index` implements Okapi BM25 from scratch (rather than using `rank_bm25`) to enable kernel-C-aware tokenisation.

8.2.1 Okapi BM25 Scoring

$$\text{score}(q,d) = \sum_{t \in q} \underbrace{\log\left(\frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} + 1\right)}_{\text{IDF}(t)} \cdot \frac{\text{tf}(t,d) \cdot (k_1 + 1)}{\text{tf}(t,d) + k_1\left(1 - b + b \cdot \frac{|d|}{|d|}\right)} \tag{2}$$

with $k_1 = 1.5$ (TF saturation), $b = 0.75$ (length normalisation).

8.2.2 C-Aware Tokenisation

Standard whitespace tokenisation destroys C semantics: `tg_throttle_up` would split to `tg`, `throttle`, `up` — losing the discriminating token. The custom tokeniser:

- Splits on C operator boundaries (`( ) { } ; , * &`) but **not** underscores.
- Lowercases all tokens.
- Removes C stopwords (`int`, `void`, `return`, `if`, `for`, etc.) that appear in every function and carry no discriminating power.

8.2.3 Adaptive Difficulty Measurement (Two-Pass Enrichment)

The enrichment pipeline runs in two passes to produce a per-example **difficulty** score $d \in [0, 1]$:

Pass 1 — collect raw gaps:

$$\text{raw\_gap}(e) = \max_{p \in \text{positives}(e)} \text{score}(q_e, p) - \text{score}(q_e, \text{top hard negative}(e)) \tag{3}$$

Large positive gap ⇒ BM25 clearly separates positive from hard negative ⇒ the training example is "easy" for BM25. Near-zero or negative gap ⇒ very hard.

Pass 2 — percentile normalisation:

$$\text{difficulty}(e) = 1 - \frac{\text{rank}(\text{raw\_gap}(e))}{N - 1} \tag{4}$$

Rank-based percentile is used instead of min-max normalisation for robustness to outliers. The raw gap and difficulty are both stored in the enriched JSONL (the raw gap for debugging, difficulty for the curriculum scheduler).

8.3 Stage 3: Curriculum-Aware Dataset (`dataset.py`)

`TripletDataset` loads the enriched JSONL and supports *adaptive curriculum learning*: early training epochs see only easy examples at their full hard-negative ratio; hard examples are gradually introduced as training progresses.

8.3.1 Curriculum Formula

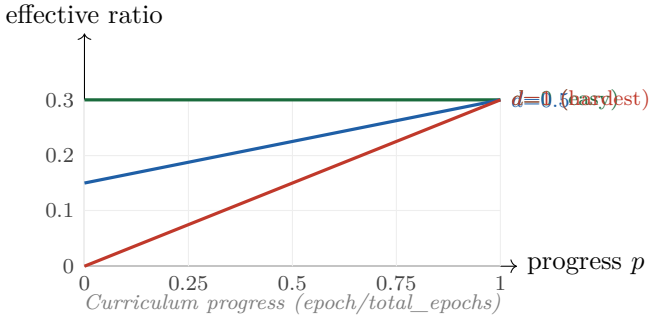
Let $p \in [0, 1]$ be curriculum progress (0 = epoch 0, 1 = final epoch) and $d \in [0, 1]$ be the difficulty of an example. The per-example effective hard ratio is:

$$\text{effective\_ratio}(p, d) = r_{\text{hard}} \cdot \big(p + (1 - p)(1 - d)\big) \tag{5}$$

where r_{hard} is the configured maximum hard ratio (default 0.3). This formula satisfies three structural guarantees:

Constraint	Value	Semantics
$f(1, \text{any})$	r_{hard}	Final epoch always uncapped (peak performance preserved)
$f(0, 0)$	r_{hard}	Easy examples see full ratio from epoch 0
$f(0, 1)$	0	Hardest examples get zero hard negatives at epoch 0
Linearity in p	Yes	No shape hyperparameter; curriculum anneals monotonically

Eq. (5) is the minimal-hyperparameter formula satisfying all three constraints simultaneously. The progression is plotted below for $r_{\text{hard}} = 0.3$:



8.3.2 Collation

`make_collate_fn()` produces `TrainingBatch` with:

- `query_input_ids` shape (B, Q_{len})
- `code_input_ids` shape $(B \cdot (1+N), C_{\text{len}})$ — positive (row 0) + negatives stacked per query
- `labels` shape $(B,)$ — all zeros (positive always at position 0; cross-entropy on this = InfoNCE)

Torch imports are *lazy* (inside `collate_fn` and the base class try/except), so the dataset and all curriculum logic is importable and testable in environments without PyTorch.

8.4 Stage 4: BiEncoder Training (`train_biencoder.py`)

8.4.1 Architecture

$$f(\text{query}) = \ell_2\text{-norm}(\text{mean-pool}(\text{CodeBERT}(\text{query\_tokens}))) \in \mathbb{R}^{768} \tag{6}$$

The query and code towers share weights (*shared bi-encoder*). Shared weights halve parameter count and typically match separate-tower performance for retrieval because query and code share enough vocabulary (function names appear in both commit messages and function names).

8.4.2 InfoNCE Loss

For a batch of B (query, positive code) pairs, with temperature τ:

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp\Big(\text{sim}(q_i, c_i^+)/\tau\Big)}{\sum_{j=1}^{B\cdot(1+N)} \exp(\text{sim}(q_i, c_j)/\tau)} \tag{7}$$

where the denominator runs over *all* code vectors in the batch: both the explicit negatives and other examples' positives (in-batch negatives, free). With $B = 32$ and $N = 7$: each query sees $32 \cdot 8 - 1 = 255$ negatives per step — sufficient for effective contrastive learning without memory banks.

8.4.3 Training Configuration

Hyperparameter	**Value / Rationale**
Backbone	`microsoft/codebert-base` (125M params, 768-dim)
Optimiser	AdamW, lr $= 2\times10^{-5}$, weight_decay $= 0.01$
LR Schedule	Linear warmup (10% steps) + cosine decay
Batch size	32 (effective 256 with gradient accumulation $\times 8$)
Epochs	3 (retrieval fine-tuning saturates quickly; more $\Rightarrow$ overfit)
Temperature τ	0.07 (following SimCLR/MoCo)
Query max tokens	128 (commit messages are short)
Code max tokens	512 (full BERT limit; function signature at start preserved)
Checkpoint metric	Val Recall@10 (save best only)

8.4.4 Curriculum Integration

At the start of each epoch:

```python
if curriculum_active:
    full_ds.set_curriculum_epoch(epoch, epochs)
    progress = full_ds._curriculum_progress
    print(f"[train] Epoch {epoch+1}: curriculum_progress={progress:.3f}"
        )
```

This propagates the epoch number into the dataset, which then computes per-example effective hard ratios according to Eq. (5) during `__getitem__`. No additional hyperparameters are needed.

9 Test Coverage

Test class	File	Tests
`TestTokenizer`	`test_training.py`	6
`TestBM25Index`	`test_training.py`	10
`TestSubsystemExtraction`	`test_training.py`	6
`TestUsefulCommitFilter`	`test_training.py`	6
`TestDateSplit`	`test_training.py`	3
`TestFunctionsAtLines`	`test_training.py`	5
`TestDifficultyGap`	`test_training.py`	5
`TestAdaptiveCurriculum`	`test_training.py`	9
Graph tests	`test_graph.py`	varies
Persistence tests	`test_persistence.py`	varies
Store integration tests	`test_store.py`	varies
Total (training module)		**50 passing**

The adaptive curriculum tests (`TestAdaptiveCurriculum`) verify all structural properties of Eq. (5): peak guarantee (final epoch always reaches r_{hard}), easy-example guarantee ($d = 0$ always sees full ratio), monotonicity in both p and d, and correct clipping to $[0, 1]$. These pass without PyTorch because all curriculum logic is pure Python.

10 Key Design Invariants

1. **CodeNode is the universal atom.** Every component — parser, graph, vector index, BM25 corpus, training dataset — operates on `CodeNode` objects. The ID scheme (`file_path::symbol_name`) is the primary key across all subsystems.

2. **Layer separation with typed edges.** Layer 1 nodes are `CodeNode` dataclasses. Layer 2 nodes are `BinarySymbol` dataclasses. Layer 3 nodes are plain dicts with `layer=3`. Cross-layer edges carry their own payload (KASLR slide, DWARF offsets). `save()` preserves only Layer 1 — the others are recomputable.

3. **Temporal split, never random.** All train/val/test splits are by commit date. Random splits allow temporal leakage. The test set corresponds to post-2023 commits, matching the gold evaluation queries.

4. **Curriculum preserves peak performance.** The curriculum formula guarantees that by the final epoch every example (including the hardest) sees the full configured r_{hard}. The curriculum only determines *when* hard examples enter full training, not *whether* they ever do.

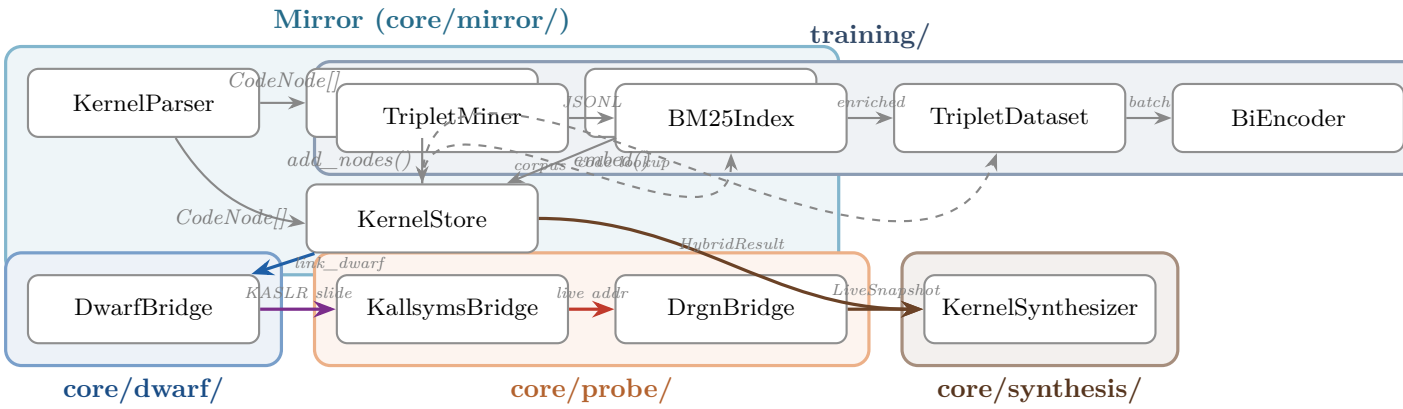
5. **Lazy imports for optional dependencies.** `torch`, `chromadb`, `drgn`, `pyelftools`, `tree_sitter` are all imported lazily (inside the methods that need them). The base classes and data models are importable without any of these installed, enabling lightweight testing and staged deployment.

6. **Graceful degradation across environments.** On macOS: drgn probe returns mock data; DWARF bridge raises `FileNotFoundError` for vmlinux; kallsyms raises `FileNotFoundError` for `/proc/kallsyms`. The static retrieval pipeline (Mirror + Synthesizer) works fully offline.

7. **Hard negative mining is separate from positive mining.** `TripletMiner` produces *only* positives. BM25 hard negative mining is a separate restartable stage. This separation means the expensive git traversal only runs once; the BM25 index can be rebuilt independently.

11 Component Interaction Diagram

The following diagram shows all major classes and their primary relationships:



12 Data Flow: Index Build vs. Query Time

12.1 Index Build (`ktalk index`)

1. `KernelParser.parse_directory()` streams `CodeNode` objects from all `*.c` and `*.h` files.
2. `KernelStore.index_nodes()` calls:
   (a) `KernelGraph.add_nodes()` — $O(N)$ node insertion with index updates.
   (b) `CodeEmbedder.embed_nodes()` — batched GPU inference.
   (c) `collection.upsert()` — write to ChromaDB on disk.
   (d) `KernelGraph.resolve_edges()` — second pass to wire all CALLS, USES_STRUCT, DEFINED_IN, INCLUDES edges.
3. `ktalk twin` (optional): loads DWARF, runs `link_dwarf()`, `link_kallsyms()`, `link_struct_layouts()`.
4. `KernelStore.save_graph()` persists Layer-1 graph to `kernel_graph.graphml`.

12.2 Query Time (`ktalk ask "..."`)

1. `KernelStore.load(storage_dir)` reloads graph from GraphML (seconds, not the expensive full reparse).
2. `KernelStore.hybrid_search(query)` performs:
   (a) Embed query with `CodeEmbedder.embed_query()`.
   (b) `collection.query()` — HNSW approximate nearest neighbour in ChromaDB.
   (c) `KernelGraph.neighborhood()` — BFS from vector seeds.
   (d) Reconstruct `CodeNode` objects from graph attributes.
3. (Optional) `DrgnBridge` probes live kernel state.
4. `KernelSynthesizer.synthesize()` builds prompt and calls LLM.
5. Streamed or batched `SynthesisResult` returned to CLI.

13 External Dependencies

Package	Used by	Purpose	Optional?	Min
`networkx`	mirror/graph	MultiDiGraph, BFS	No	3.x
`chromadb`	mirror/store	Vector index	No	0.4+
`numpy`	mirror/embedder	Embedding arrays	No	1.24+
`sentence-transformers`	mirror/embedder	CodeBERT inference	No	2.x
`tree-sitter`	mirror/parser, training/mine	C AST	No	0.22+
`tree-sitter-c`	mirror/parser, training/mine	C grammar	No	0.21+
`pyelftools`	dwarf/bridge	DWARF parsing	Yes	0.29+
`drgn`	probe/drgn_bridge	Live memory	Yes	0.0.23
`torch`	training/*	BiEncoder training	Yes	2.0+
`transformers`	training/*	CodeBERT tokenizer	Yes	4.30+
`tiktoken`	synthesis	Token estimation	Yes	0.5+
`ollama`	synthesis	Local LLM (fallback HTTP)	Yes	0.1+
`openai`	synthesis	OpenAI / vLLM	Yes	1.0+
`anthropic`	synthesis	Claude API	Yes	0.20+

All optional dependencies are guarded by `try/except ImportError` and imported lazily within the methods that need them. The core Mirror pipeline (parse + embed + index + query) requires only the first five packages.

A Graph Edge Type Reference

Edge type	Source node	Target node	Created by	Payload
`CALLS`	function	function	`resolve_edges()`	—
`USES_STRUCT`	function	struct/union	`resolve_edges()`	—
`DEFINED_IN`	any symbol	file	`resolve_edges()`	—
`INCLUDES`	file	file	`resolve_edges()`	—

Edge type	Source node	Target node	Created by	Payload
`RELATED_TO`	any	any	(query-time, soft)	—
`SOURCE_TO_BINARY`	CodeNode	BinarySymbol	`link_dwarf()`	`dwarf_addr_start`, `sec`
`BINARY_TO_LIVE`	BinarySymbol	KallsymNode	`link_kallsyms()`	`kaslr_slide`, `live_add`
`FIELD_TO_OFFSET`	struct CodeNode	field node	`link_struct_layouts()`	`byte_offset`, `byte_siz`

B Key Bug Fixes (F-series)

ID	File	Issue	Fix
F-5	graph.py	O(N^2) INCLUDES resolution	Suffix index → O(1)
F-6	graph.py	Layer-2/3 nodes crash GraphML save	Layer-1-only serialisation
F-8	graph.py	Symbol index missing	`_symbol_index` maintaine
F-11	store.py	Docstring not stored in ChromaDB	Added to `to_metadata()`
F-12	store.py	Context duplicates primary hits	Dedup via `primary_ids` s
F-13	graph.py	No schema version tracking	`schema_version` in Graph
F-14	dwarf/bridge.py	Stale DWARF cache on mtime collision	mtime + size + CRC32 k
F-16	dwarf/bridge.py	Anonymous unions invisible	Recursive `_flatten_anon`
F-17	probe/kallsyms.py	`do_fork` removed in v5.7	`kernel_clone` added to a
F-18	probe/kallsyms.py	`sys_read` renamed on x86-64	Both variants in anchor li
F-20	probe/drgn_bridge.py	`task.state` renamed to `task.__state` in v5.14	Try both
F-22	probe/drgn_bridge.py	`cpu_rq` ImportError crashes	Separate import guards
F-24	synthesis/synthesizer.py	Fixed char limits ignore context window	Token-budget-aware alloca
F-28	cli/ktalk.py	`__import__` antipattern	Static imports at module

Document generated April 18, 2026. The system is under active development.
Training pipeline (Stages 1–4) is complete and tested.
Cross-encoder reranker (Stage 5) and hardware setup are planned next.